



nice to meet u:)





4 指令系统

计算机在执行的指令长什么样？
程序又是怎样靠这些指令运行起来的？

5 CPU

CPU是怎样一步一步执行指令，并指挥整个计算机工作的？

3 存储器

程序和数据平时放在哪里？
计算机需要时又是怎样找到它们的？

6 总线

计算机各个部件之间是怎样传信息的？

2 数据的表示与运算

数据在计算机里怎样存，计算机怎样拿它们来运算？

7 输入/输出系统

计算机怎样和外部设备连接、交换信息？
用什么方式完成输入输出操作？

1 概述

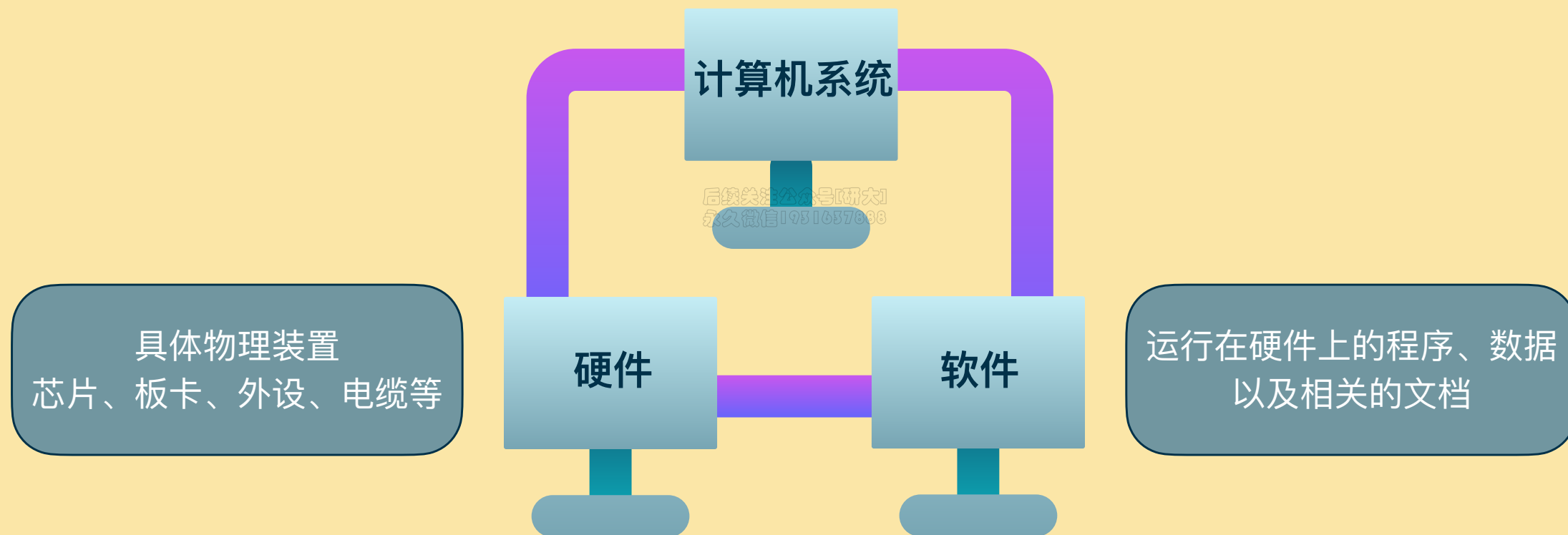
计算机系统由什么组成？怎样运行程序？

计算机组成原理





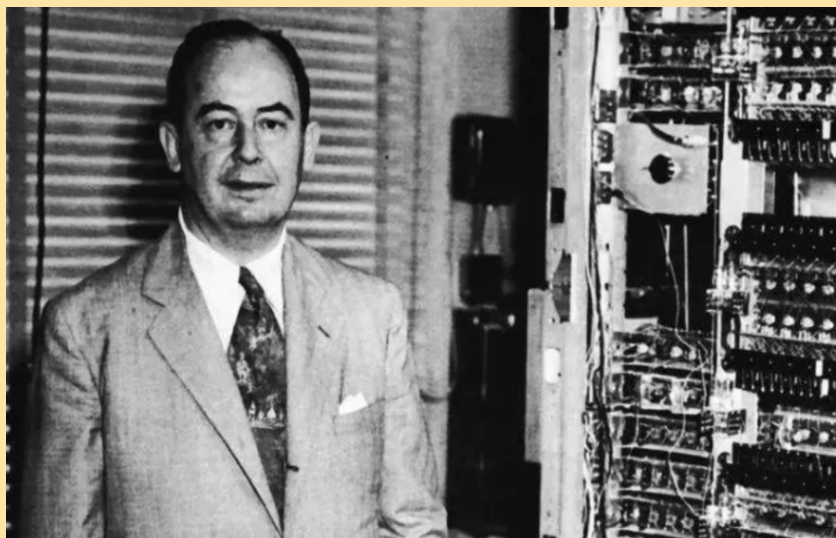
1. 计算机系统的基本组成





1. 计算机系统的基本组成

1.1 计算机硬件



①冯·诺依曼结构计算机

(1) 采用“**存储程序**”的工作方式：将编制好的程序和初始数据预先存入主存储器，计算机启动后能自动连续取指令并执行，直至程序结束，无须人工干预

(2) 计算机由**运算器、控制器、存储器、输入设备、输出设备**5个基本部件构成

(3) 存储器不仅能存放数据，而且也能存放指令，**形式上数据和指令没有区别**，但计算机应能区分它们

(4) 计算机内部以**二进制**形式表示指令和数据

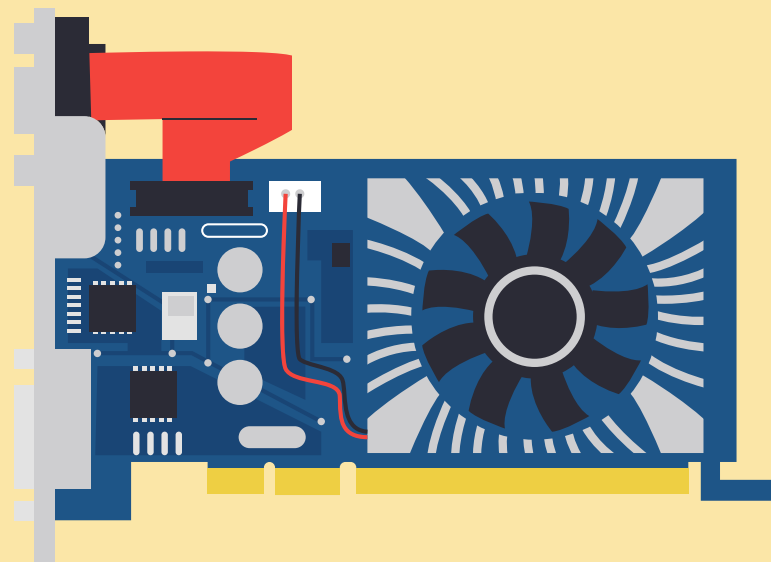
(5) 指令由**操作码和地址码**组成，其中操作码指明操作类型，地址码指出操作数的地址





1. 计算机系统的基本组成

1.1 计算机硬件



中央处理器CPU

- 后续关注CSDN公众号
永久微信1931637888
- CPU是整个计算机的核心部件，主要用于指令的执行
 - CPU主要包含两种基本部件：**数据通路**和**控制器**
 - 数据通路主要包含**运算逻辑部件**和**通用寄存器**等，其功能是用来**执行算数和逻辑运算**等操作
 - 控制器用来**对指令进行译码**，生成相应的**控制信号**，以控制数据通路进行正确的操作

②计算机的功能部件

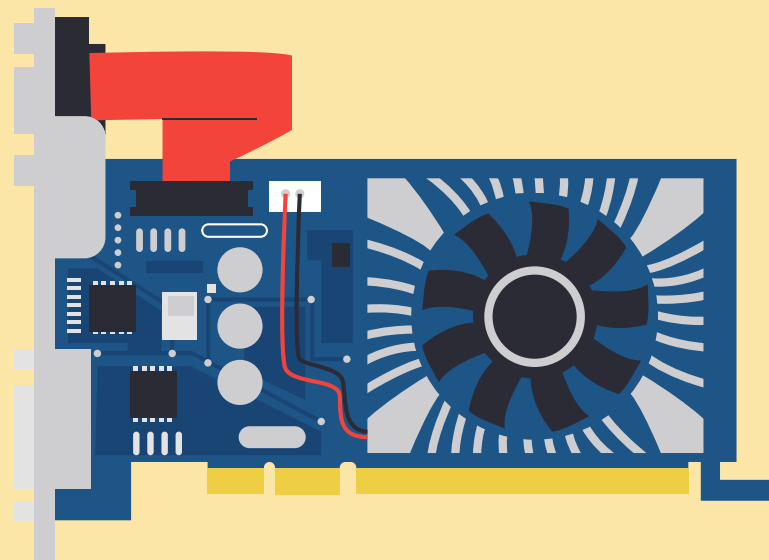
将在**第5章**进行深入学习~_~





1. 计算机系统的基本组成

1.1 计算机硬件



存储器

后续关注
永久微信1931637888

• 存储器分为**内存**和**外存**

- 内存包括**主存储器**（简称主存）和**高速缓存**（cache）
- 外存包括**辅助存储器**和海量后备存储器，通常把系统运行时直接和主存交换信息的存储器称为辅助存储器（简称辅存），目前主要的辅助存储器是磁盘存储器和固态硬盘

②计算机的功能部件

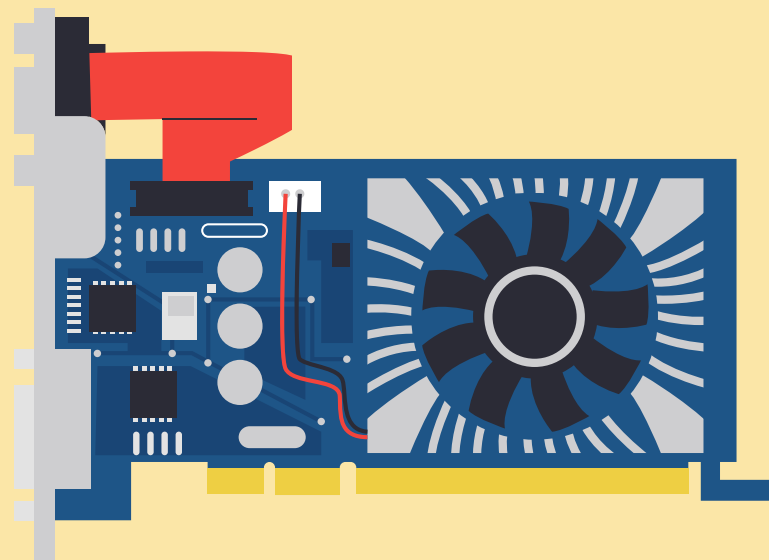
将在**第3章**进行深入学习~ω~





1. 计算机系统的基本组成

1.1 计算机硬件



外部设备

- 后续关注：微信公众号1931637888
- 外部设备简称**外设**，也称为**I/O设备**
 - 外设通常由**机械部分**和**电子部分**组成，通常机械部分是外设本身，而电子部分是控制外部设备的**I/O控制器**或**I/O适配器**（I/O控制器或I/O适配器统称为**设备控制器**）
 - 外设通过**设备控制器**连接到主机上

②计算机的功能部件

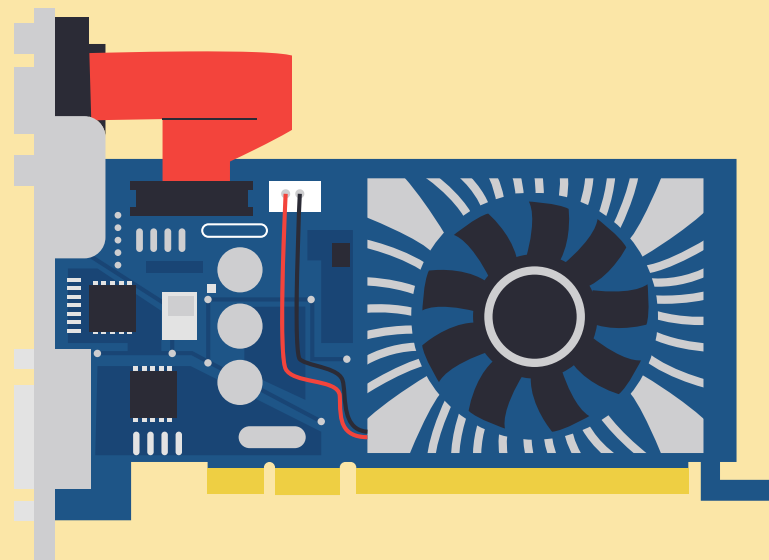
将在**第7章**进行深入学习~ω~





1. 计算机系统的基本组成

1.1 计算机硬件



总线

- 总线是传输信息的介质，用于在部件之间传输信息
- CPU、主存和I/O模块通过总线互连，在CPU和I/O模块内部都内含相应的存储部件（缓存器），用于配合信息传输

将在第6章进行深入学习~ω~

②计算机的功能部件





1. 计算机系统的基本组成

1.2 计算机软件



- 为有效、安全地使用和管理计算机以及为开发和运行应用软件而提供的各种软件
- 介于计算机硬件与应用程序之间

- 专门为数据处理、科学计算、事务管理、多媒体处理等应用所编写的各类程序都称为应用软件
- 操作系统（如Windows）
- 语言处理系统（如C语言编译器）
- 数据库管理系统（如Oracle）
- 各类实用程序（如磁盘碎片整理、备份程序）





1. 计算机系统的基本组成

1.3 软件和硬件的逻辑功能等价性

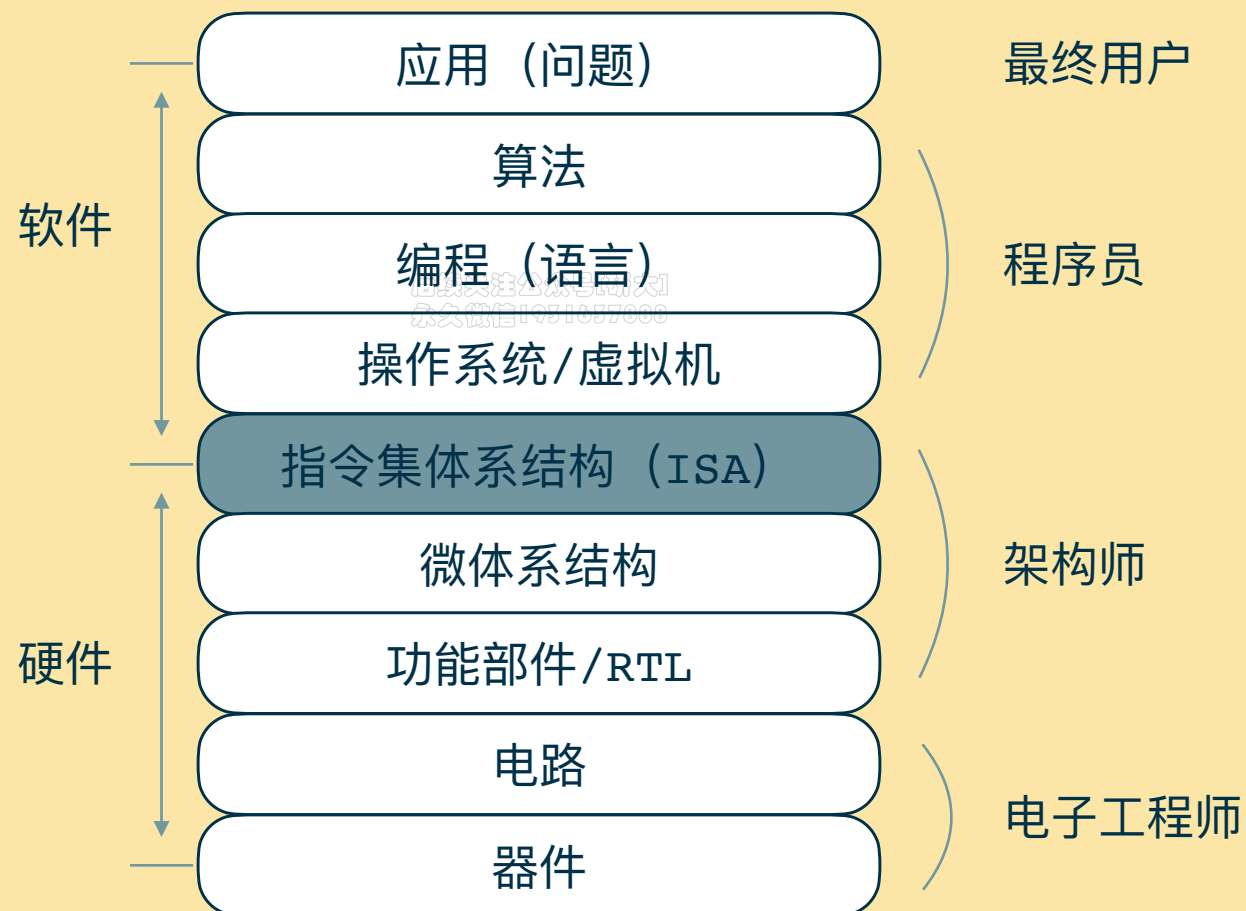
同一功能，既可以用硬件实现，也可以用软件实现
在相同规范下，二者在逻辑功能上是**等价**的，但硬件通常更快





2. 计算机系统的层次结构

2.1 计算机系统抽象层的转换





2. 计算机系统的层次结构

2.1 计算机系统抽象层的转换

高级语言与低级语言

高级语言 一种机器无关语言
更接近人的思维方式
更强调程序逻辑本身，而不是具体机器如何做

低级语言 也称为机器级语言
与运行程序的计算机底层结构密切相关
包括**机器语言**和**汇编语言**

机器语言 用二进制编码表示的机器指令序列
可读性差、不容易记忆、编写困难、阅读困难

汇编语言 用简短的英文符号去表示机器指令
与机器语言建立对应关系
是机器语言的符号化表示





2. 计算机系统的层次结构

2.1 计算机系统抽象层的转换

翻译程序

- (1) **汇编程序** 也称汇编器
用来将汇编语言源程序翻译成机器语言目标程序
- (2) **解释程序** 也称解释器
用来将源程序中的语句按其执行顺序逐条翻译成机器指令并立即执行
- (3) **编译程序** 也称编译器
用来将高级语言程序翻译成汇编语言或机器语言目标程序





2. 计算机系统的层次结构

2.1 计算机系统抽象层的转换

ISA

ISA（指令集体系结构）是软件和硬件之间接口的一个完整定义

ISA定义了一台计算机可以执行的全部指令集合

每条指令规定了计算机执行什么操作，所处理的操作数存放的地址空间以及操作数类型





2. 计算机系统的层次结构

2.1 计算机系统抽象层的转换

ISA

- 数据类型及格式
- 指令格式
- 寻址方式
- 可访问地址空间大小
- 程序可访问的寄存器个数、位数、编号
- 控制寄存器的定义
- I/O空间的编址方式
- 中断结构
- 机器工作状态的定义和切换
- 输入输出结构
- 数据传送方式
- 存储保护方式

后续关注公众号「研大」
永久微信1951657888

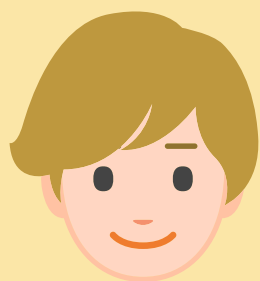
ISA（指令集体系结构）是软件和硬件之间接口的一个完整定义



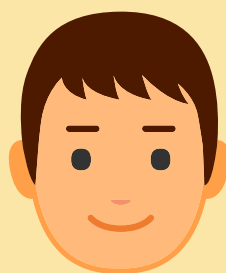


2. 计算机系统的层次结构

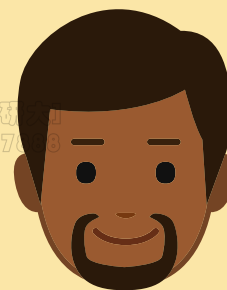
2.2 计算机系统的不同用户



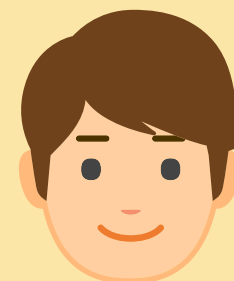
①最终用户



②系统管理员



③应用程序员



④系统程序员

后续关注公众号
永久微信1951657053





2. 计算机系统的层次结构

2.2 计算机系统的不同用户



①最终用户

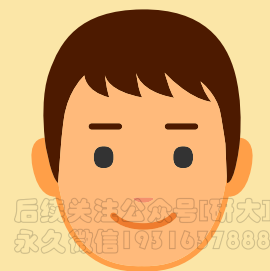
使用应用程序完成特定任务的计算机用户称为最终用户
他们通过操作系统提供的界面与计算机交互，无须了解底层技术细节





2. 计算机系统的层次结构

2.2 计算机系统的不同用户



②系统管理员

利用操作系统等软件提供的功能对系统进行配置、管理和维护
以建立高效合理的系统环境供计算机用户使用的操作人员





2. 计算机系统的层次结构

2.2 计算机系统的不同用户



③应用程序员

使用高级编程语言编制应用程序的程序员





2. 计算机系统的层次结构

2.2 计算机系统的不同用户



④系统程序员

设计和开发系统软件的程序员

需要熟悉计算机底层的相关硬件和系统结构

甚至可能需要直接与计算机硬件和指令系统打交道





2. 计算机系统的层次结构

2.3 透明

在计算机技术中，一个存在的事物或概念从某个角度看似乎不存在
即对实际存在的事物或概念感知不到，称为**透明**





3. 程序开发与执行过程

3.1 从源程序到可执行程序

```
#include<stdio.h>
int main()
{
    printf("hello, world\n");
}
```

计算机如何执行这个C语言程序？





```
#include<stdio.h>
int main()
{
    printf("hello, world\n");
}
```

3. 程序开发与执行过程

3.1 从源程序到可执行程序

Step1.通过程序编辑软件得到hello.c文件

#	i	n	c	l	u	d	e	<sp>	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	<sp>	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	<sp>	<sp>	<sp>	<sp>	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	<sp>	w	o	r	l	d	\	n	"	)	;	\n	}
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	125

hello.c在计算机中以ASCII字符方式存放





```
#include<stdio.h>
int main()
{
    printf("hello, world\n");
}
```

3. 程序开发与执行过程

3.1 从源程序到可执行程序

Step2.将hello.c进行预处理、编译、汇编和链接
最终生成可执行目标文件

后续关注公众号「研大」
公众号ID:19511457888

```
unix > gcc -o hello hello.c
```

用GCC编译驱动程序进行处理

unix > : shell命令行解释器的命令行提示符

gcc : GCC编译驱动程序名

-o : 表示后面为输出文件名

hello.c : 要处理的源程序





3. 程序开发与执行过程

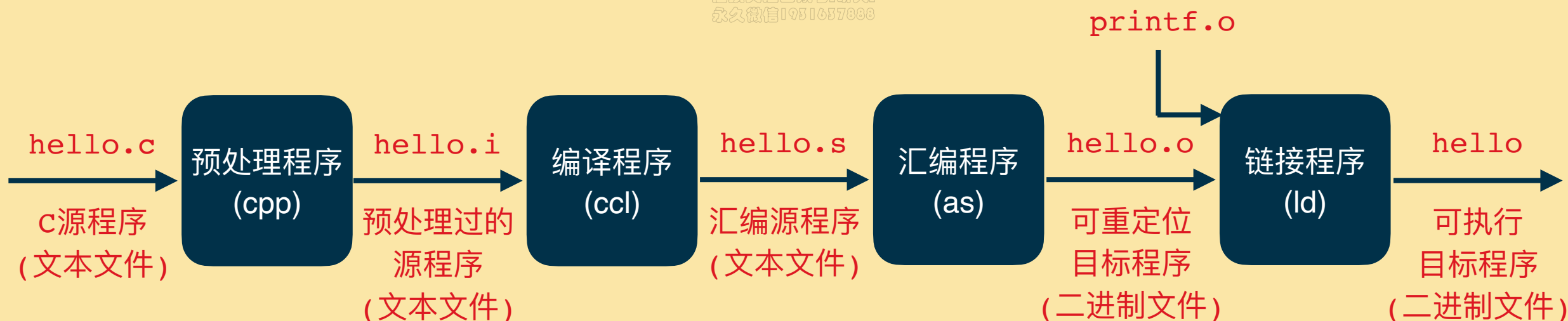
3.1 从源程序到可执行程序

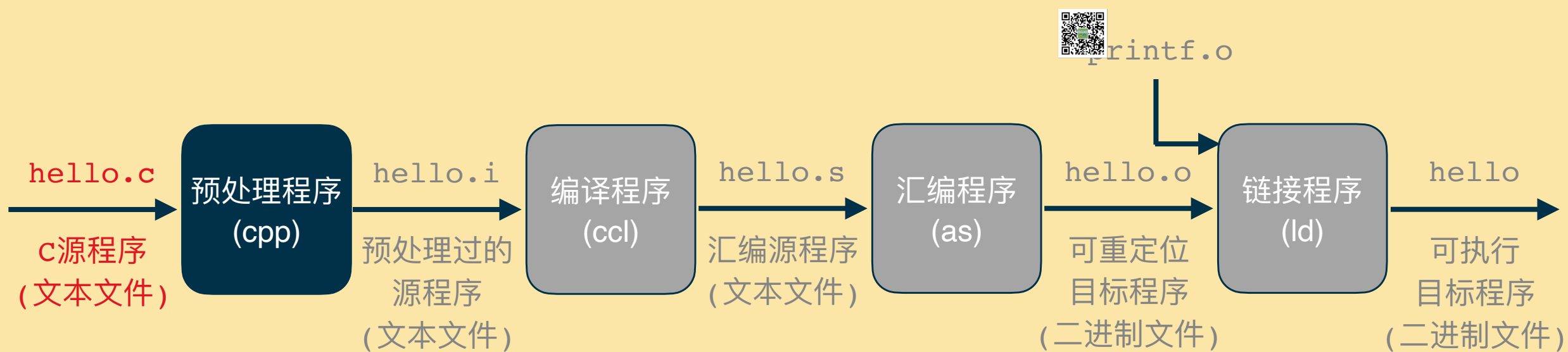
```
#include<stdio.h>
int main()
{
    printf("hello, world\n");
}
```

```
unix > gcc -o hello hello.c
```

Step2. 将hello.c进行预处理、编译、汇编和链接
最终生成可执行目标文件

后续关注公众号[研大]
永久微信1951657888



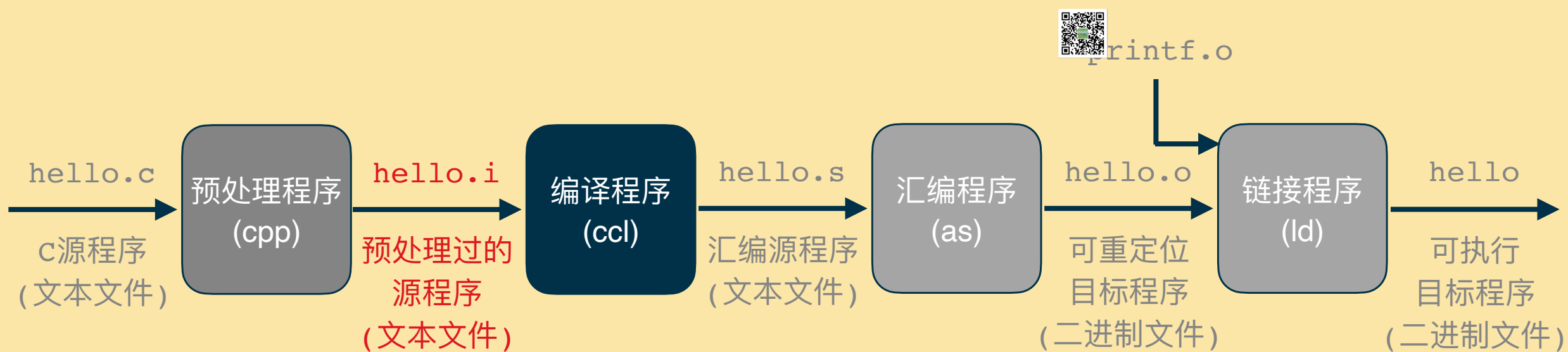


后续关注公众号[研大]
永久微信1951657888

①预处理阶段

预处理程序 (`cpp`) 对源程序中以字符 `#` 开头的命令进行处理
例如将 `#include` 命令后面的 `.h` 文件内容嵌入到源程序文件中
预处理程序的输出结果还是一个源程序文件，以 `i` 为扩展名



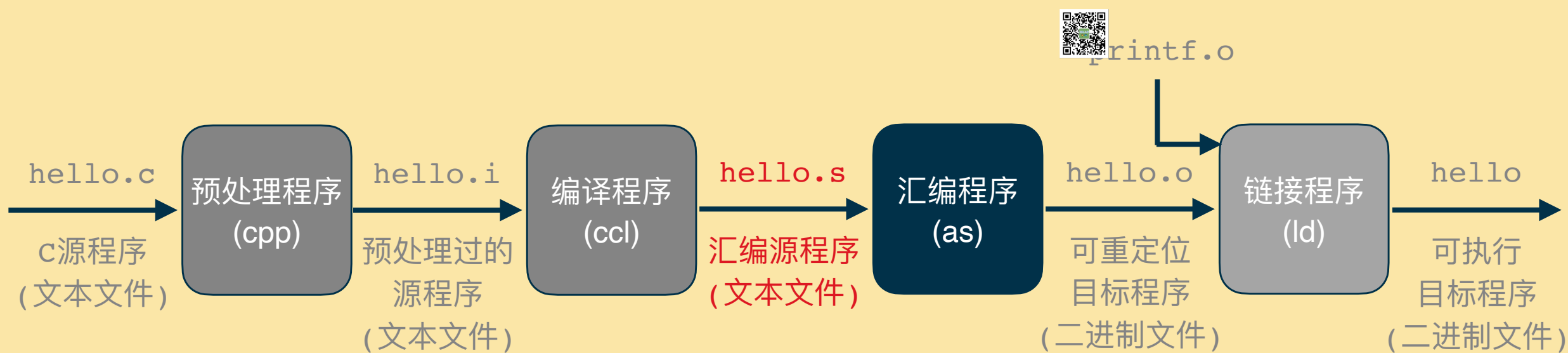


后续关注公众号「研大」
永久微信1951657888

②编译阶段

编译程序(cc1)对预处理后的源程序进行编译
生成一个汇编语言源程序文件，以s为扩展名



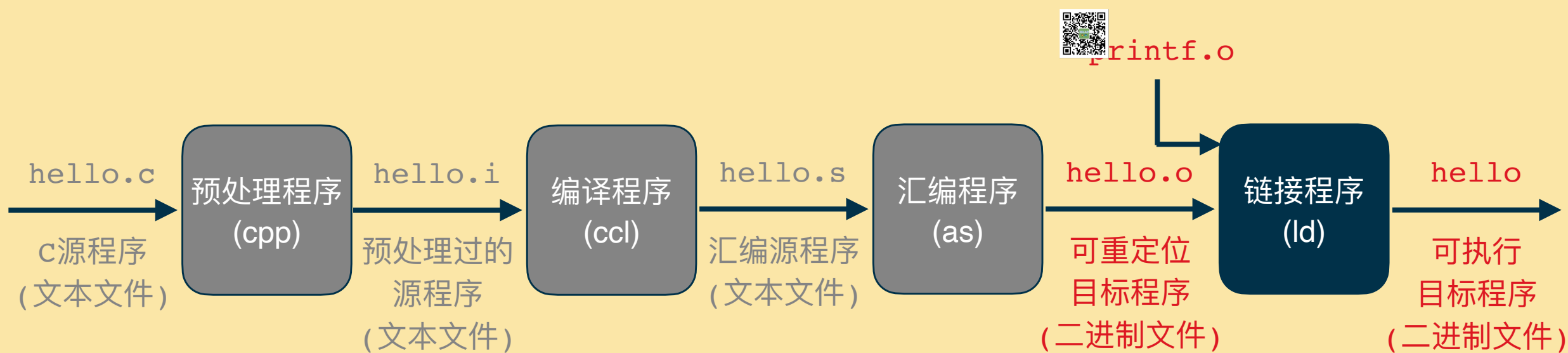


后续关注公众号「研大」
永久微信1951657888

③汇编阶段

汇编程序 (as) 对汇编语言源程序进行汇编
生成一个**可重定位目标文件**，以o为扩展名
它是一种**二进制文件**，其中的代码是机器指令，数据和其他信息也用二进制表示
所以它是**不可读的**，打开显示乱码





后续关注公众号[研大]
永久微信1951657888

④链接阶段

链接程序(ld)将多个可重定位目标文件和标准库函数合并为一个可执行目标文件

本例中，链接器将hello.o和标准库函数printf所在的可重定位目标模块printf.o进行合并

生成可执行目标文件hello

最终生成的可执行文件被保存在磁盘上





3. 程序开发与执行过程

3.2 可执行文件的启动和执行



存放在磁盘中的
可执行目标文件hello

后续关注公众号
永久微信1951655566

```
unix > ./hello
```

通过shell命令行解释器启动执行





3. 程序开发与执行过程

3.2 可执行文件的启动和执行



存放在磁盘中的
可执行目标文件hello

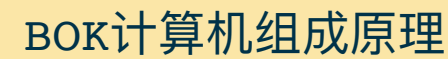
后续关注公众号
永久微信1951655566

```
unix > ./hello  
hello,world  
unix >
```

通过shell命令行解释器启动执行

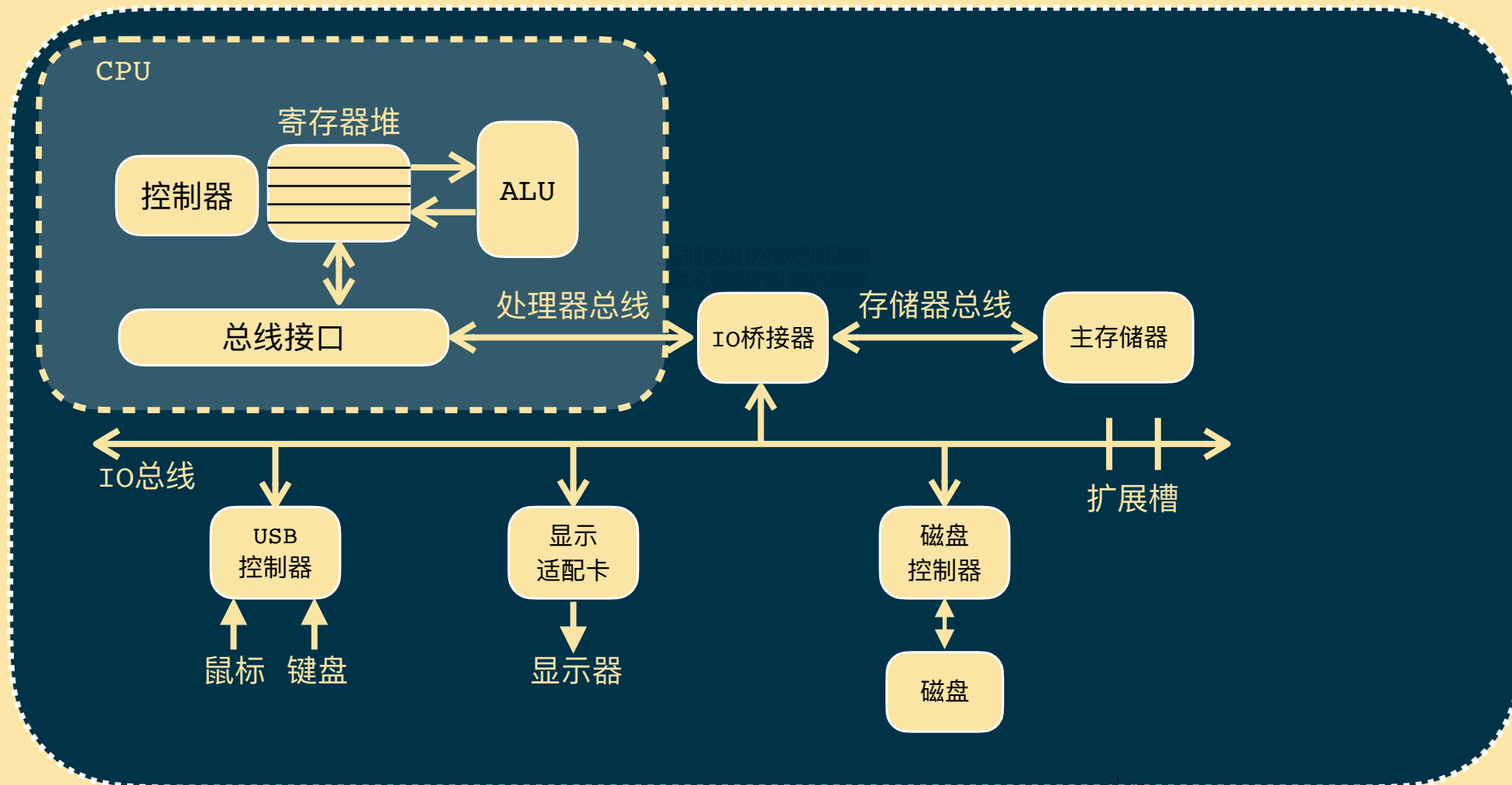


```
unix > ./hello
hello, world
unix >
```



3. 程序开发与执行过程

3.2 可执行文件的启动和执行

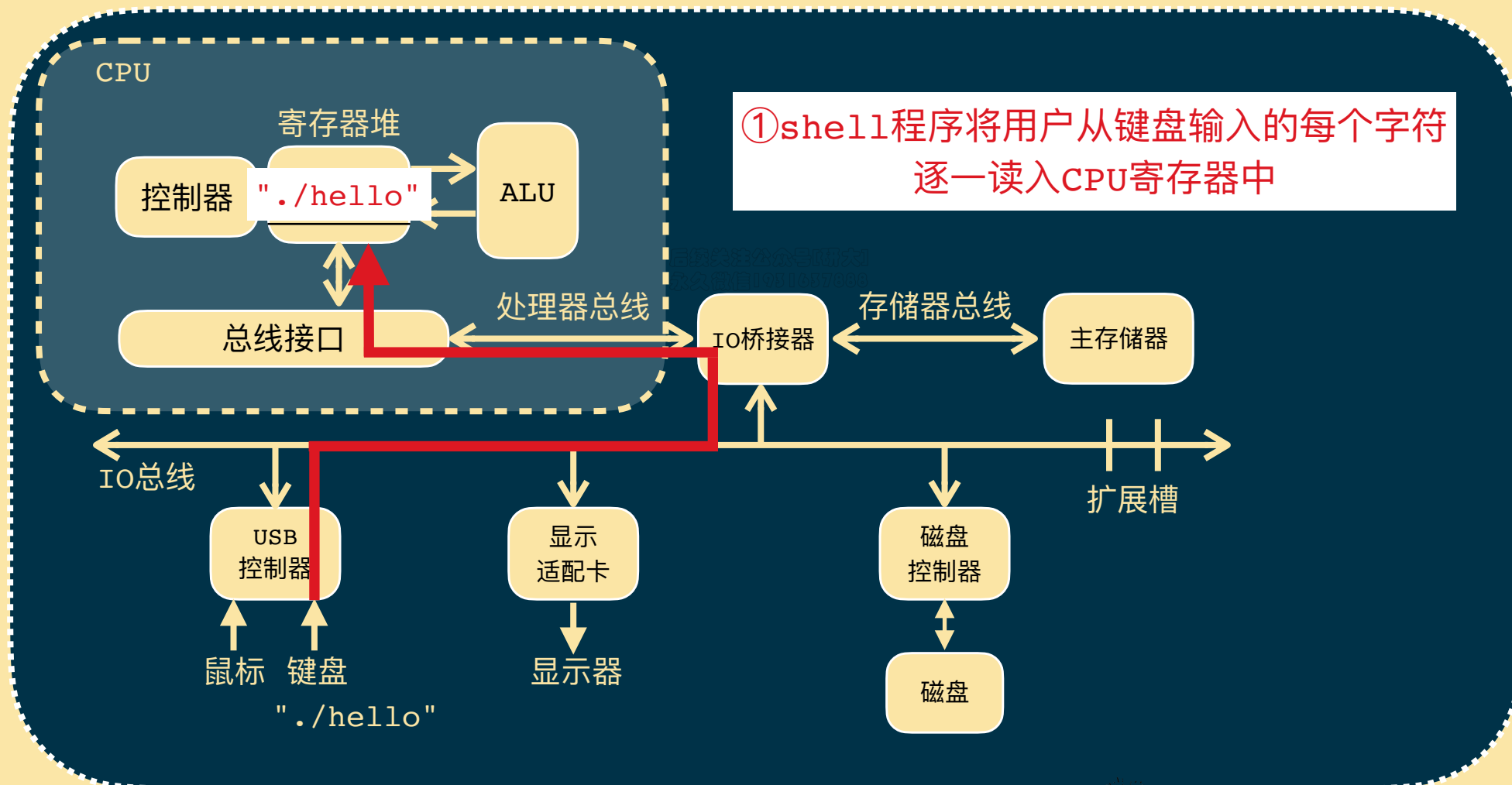


```
unix > ./hello
hello, world
unix >
```



3. 程序开发与执行过程

3.2 可执行文件的启动和执行

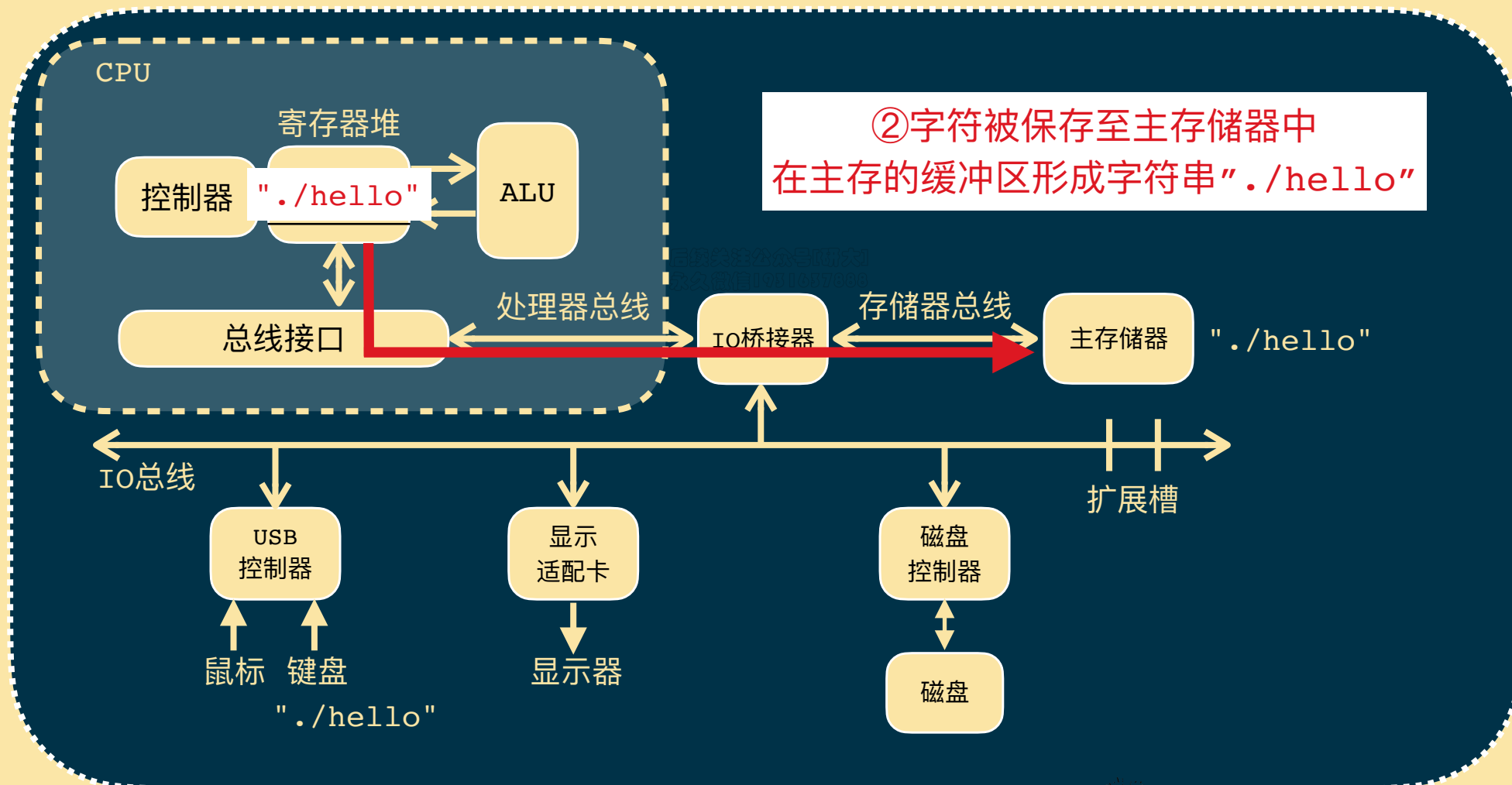


```
unix > ./hello
hello, world
unix >
```



3. 程序开发与执行过程

3.2 可执行文件的启动和执行

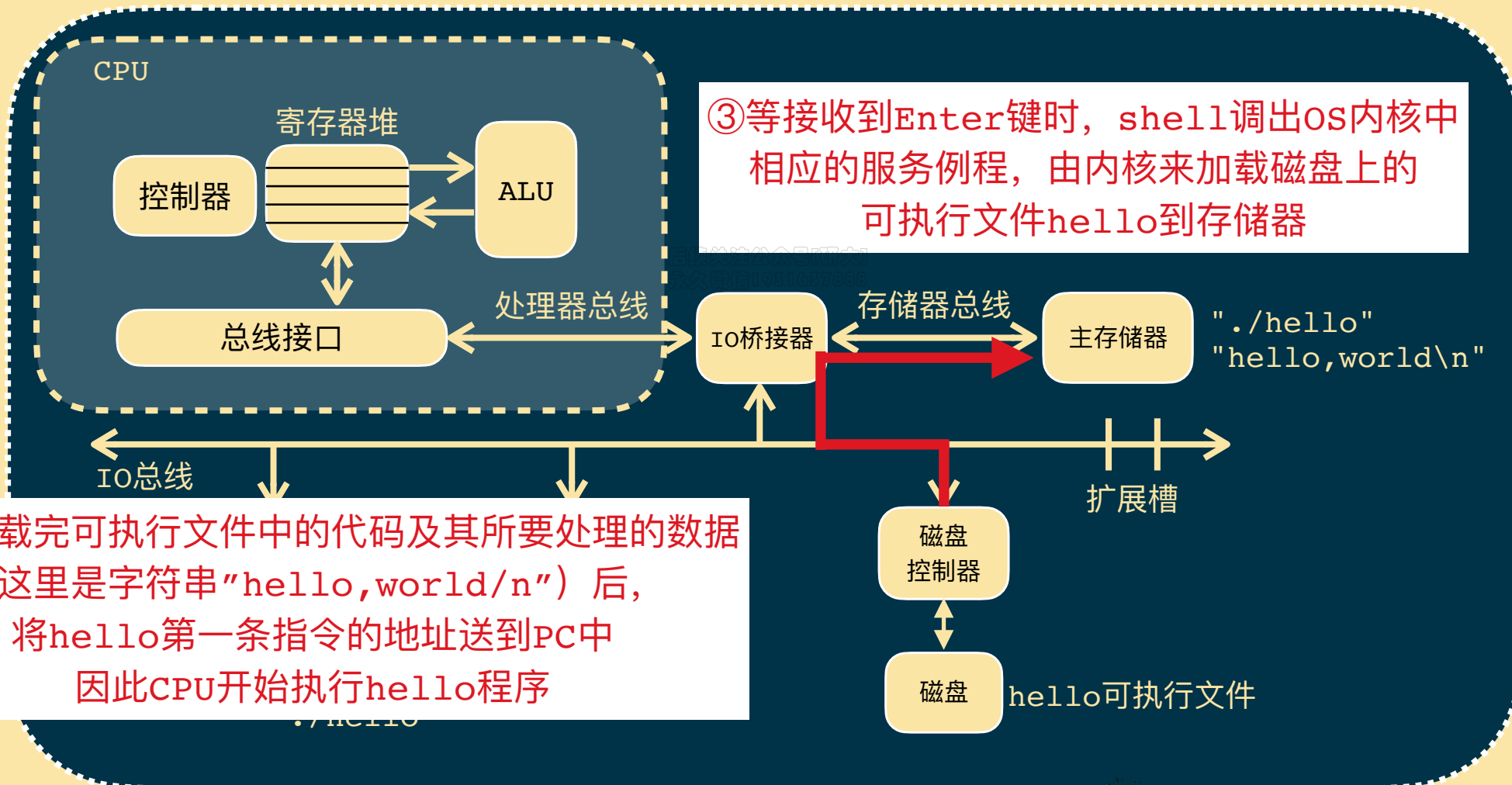


```
unix > ./hello
hello, world
unix >
```



3. 程序开发与执行过程

3.2 可执行文件的启动和执行

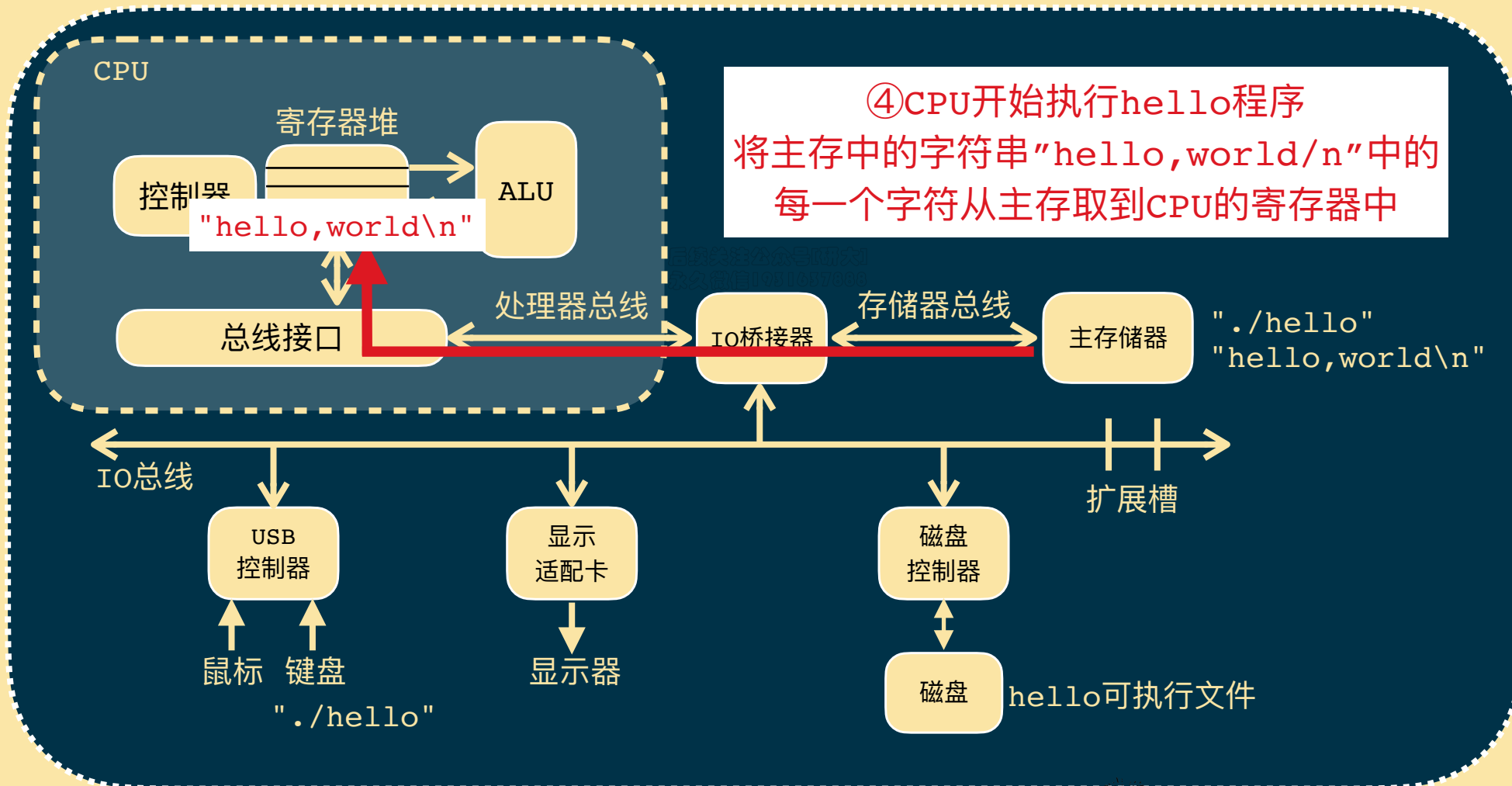


```
unix > ./hello
hello, world
unix >
```



3. 程序开发与执行过程

3.2 可执行文件的启动和执行

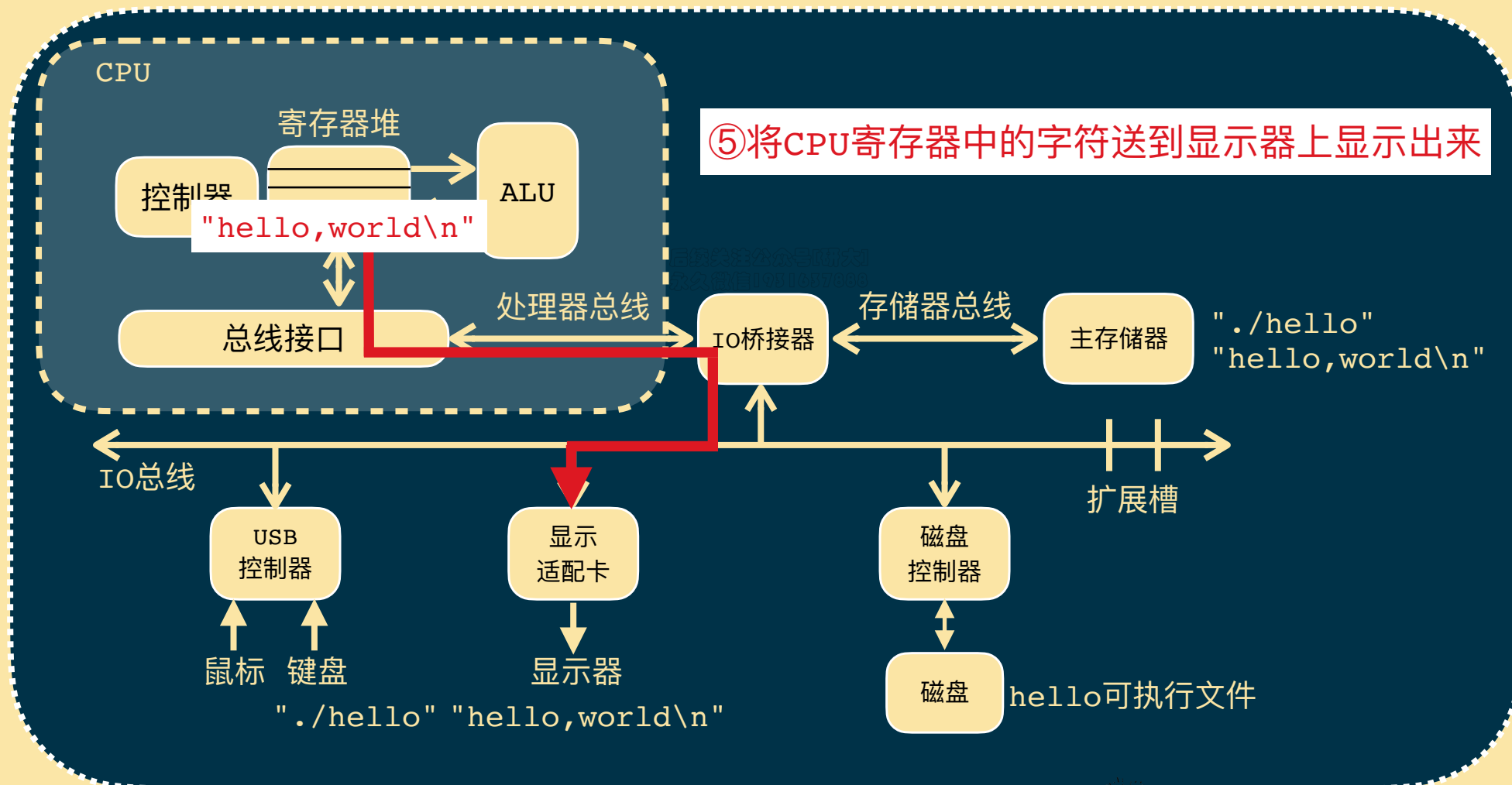


```
unix > ./hello
hello, world
unix >
```



3. 程序开发与执行过程

3.2 可执行文件的启动和执行





3. 程序开发与执行过程

3.3 程序和指令的关系



可执行文件的执行实际上是
所包含的机器代码段执行的过程

hello可执行目标文件的组成





3. 程序开发与执行过程

3.3 程序和指令的关系

ELF头

代码段(.text)

只读数据段(.rodata)

数据段(.data)

BSS段(.bss)

符号表

重定位信息

动态链接相关信息

节头表/程序头表

hello可执行目标文件的组成

```
31 ed 49 89 d1 5e 48 89 e2 48 83 e4 f0 50 54 45
31 c0 31 c9 48 8d 3d ce 00 00 00 ff 15 4f 2f 00
00 f4 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00
48 8d 3d 91 2f 00 00 48 8d 05 8a 2f 00 00 48 39
```

hello可执行目标文件中的代码段.text前64字节展示





```
31 ed 49 89 d1 5e 48 89 e2 48 83 e4 f0 50 54 45
31 c0 31 c9 48 8d 3d ce 00 00 00 ff 15 4f 2f 00
00 f4 66 2e 0f 1f 84 00 00 00 00 0f 1f 40 00
48 8d 3d 91 2f 00 00 48 8d 05 8a 2f 00 00 48 39
```

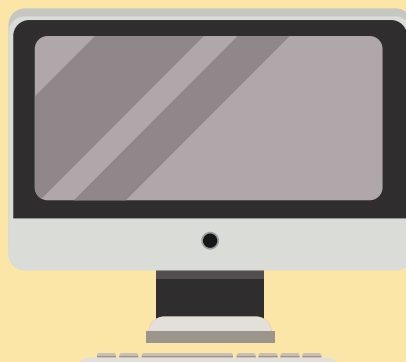
hello可执行目标文件中的代码段.text前64字节展示

3. 程序开发与执行过程

3.3 程序和指令的关系

31 ed (00110001 11101101)

机器指令



把ebp寄存器清零





3. 程序开发与执行过程

3.3 程序和指令的关系

```
31 ed 49 89 d1 5e 48 89 e2 48 83 e4 f0 50 54 45
31 c0 31 c9 48 8d 3d ce 00 00 00 ff 15 4f 2f 00
00 f4 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00
48 8d 3d 91 2f 00 00 48 8d 05 8a 2f 00 00 48 39
```

hello可执行目标文件中的代码段.text前64字节展示



先把xx的数据地址放进寄存器
然后把xx寄存器清零
再调用xx输出函数
然后把控制权转移到下一段代码

...



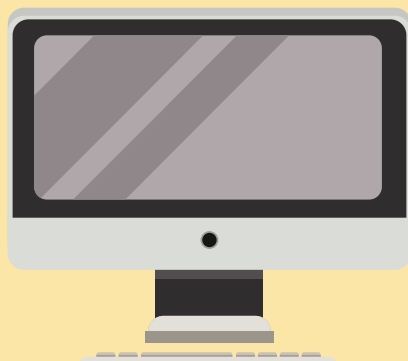


3. 程序开发与执行过程

3.3 程序和指令的关系

```
31 ed 49 89 d1 5e 48 89 e2 48 83 e4 f0 50 54 45
31 c0 31 c9 48 8d 3d ce 00 00 00 ff 15 4f 2f 00
00 f4 66 2e 0f 1f 84 00 00 00 00 00 0f 1f 40 00
48 8d 3d 91 2f 00 00 48 8d 05 8a 2f 00 00 48 39
```

hello可执行目标文件中的代码段.text前64字节展示



先把xx的数据地址放进寄存器
然后把xx寄存器清零
再调用xx输出函数
然后把控制权转移到下一段代码

...

hello程序在底层实际上会被拆解成
很多条机器指令
再被CPU一条一条执行





```
31 ed 49 89 d1 5e 48 89 e2 48 83 e4 f0 50 54 45
31 c0 31 c9 48 8d 3d ce 00 00 00 ff 15 4f 2f 00
00 f4 66 2e 0f 1f 84 00 00 00 00 0f 1f 40 00
48 8d 3d 91 2f 00 00 48 8d 05 8a 2f 00 00 48 39
```

hello可执行目标文件中的代码段.text前64字节展示

3. 程序开发与执行过程

3.3 程序和指令的关系

31 ed (00110001 11101101)

机器指令



操作码

寻址特征

地址字段

CPU之所以能知道机器指令在干嘛
是因为它符合当前指令系统规定好的指令格式
即CPU可以拆解出这一串二进制代码的哪部分代表什么含义





3. 程序开发与执行过程

3.3 程序和指令的关系



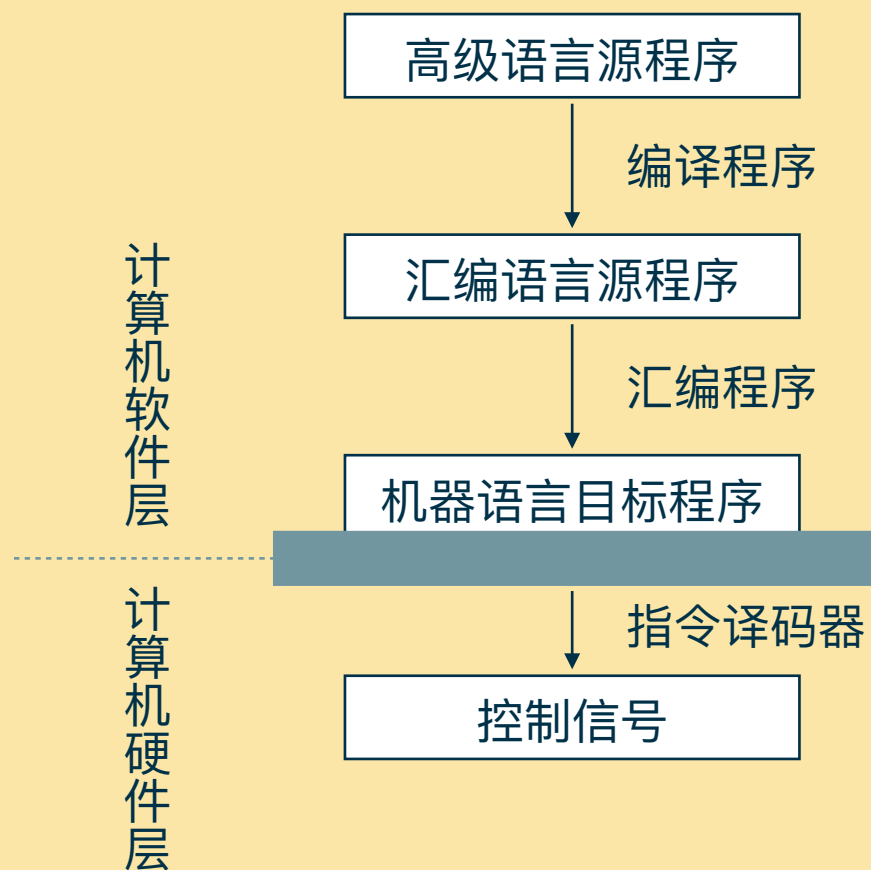
可执行目标文件中的机器代码段，本质上就是一条条已经编码好的机器指令
程序运行时，CPU并不是直接看懂高级语言源代码，而是不断从机器代码段中取出一串串二进制编码
按照既定的指令格式对其进行拆解和解释，从而确定其所表示的具体操作，并依次加以执行





3. 程序开发与执行过程

3.3 程序和指令的关系



```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

后续关注微信公众号
永久微信1951157098

```
lw $15,0($2)  
lw $16,4($2)  
sw $16,0($2)  
sw $15,4($2)
```

```
1000 1100 0100 1111 0000 0000 0000 0000  
1000 1100 0101 0000 0000 0000 0000 0100  
1010 1100 0101 0000 0000 0000 0000 0000  
1010 1100 0100 1111 0000 0000 0000 0100
```

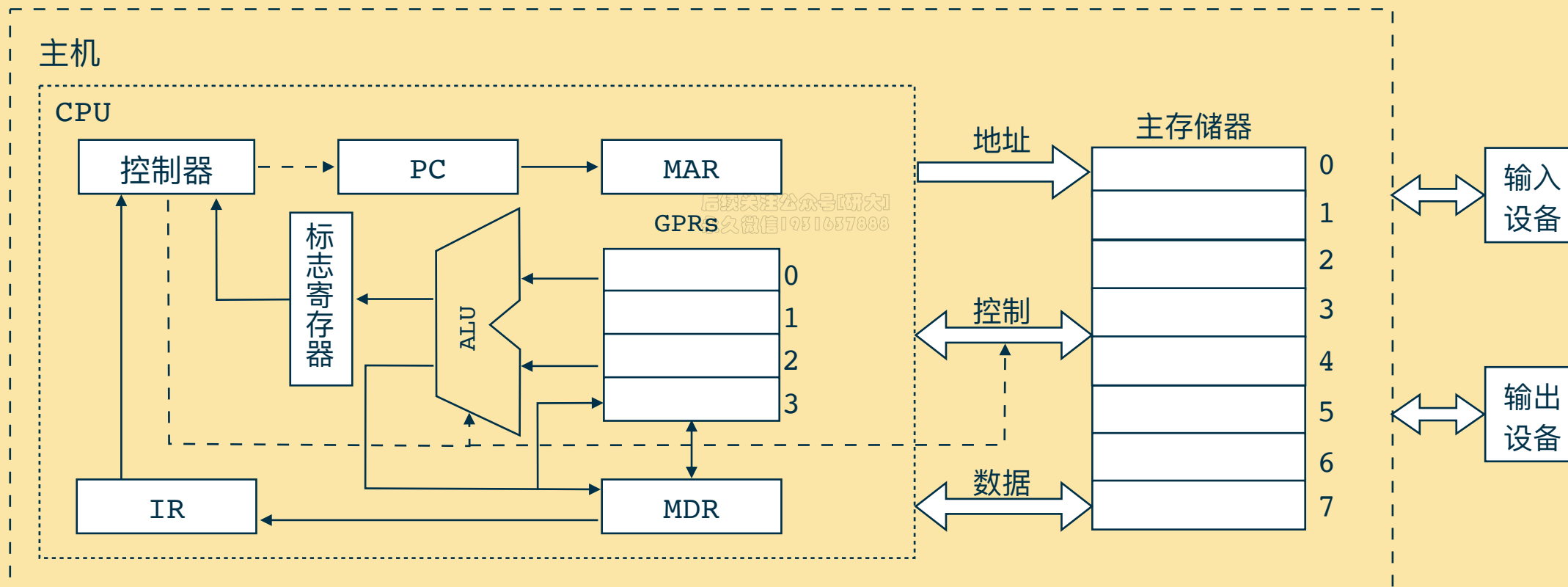
```
..., EXT0p=1,ALUSelA=1,ALUSelB=11,ALUOp=add,  
lorD=1,Read,MemtoReg=1,RegWr=1, ...
```





3. 程序开发与执行过程

3.4 指令的执行过程



冯·诺依曼结构模型机





3. 程序开发与执行过程

3.4 指令的执行过程

①根据PC取指令到IR

将PC的内容送MAR，MAR中的内容直接送地址线，同时控制器将读信号送读/写信号线
主存根据地址线上的地址和读信号，从指定存储单元开始读出指令，并送到数据线上
MDR从数据线接受指令信息，并传送到IR中

②指令译码并送出控制信号

控制器根据IR中指令的操作码，生成相应的控制信号，送到不同的执行部件
例如，若IR中的指令是ALU运算指令，则控制信号被送到ALU的操作控制端，以控制ALU将要进行什么运算

③取操作数

如果是ALU运算类指令
则根据IR中指令指定的寄存器编号选择两个或一个寄存器中的内容作为操作数，送到ALU的输入端

④指令执行

若是ALU运算类指令，则ALU在控制信号的控制下，进行相应的运算，并生成运算结果和相应的标志信息
若是取数/存数指令，则主存在读/写控制信号的控制下，对地址线指定的存储单元进行读/写

⑤回写结果

将ALU运算的结果写入指定编号的寄存器或指定的主存单元中





①根据PC取指令到IR

将PC的内容送MAR，MAR中的内容直接送地址线，同时控制器将读信号送读/写信号线
主存根据地址线上的地址和读信号，从指定存储单元开始读出指令，并送到数据线上
MDR从数据线接受指令信息，并传送到IR中

②指令译码并送出控制信号

控制器根据IR中指令的操作码，生成相应的控制信号，送到不同的执行部件

例如，若IR中的指令是ALU运算指令，则控制信号被送到ALU的操作控制端，以控制ALU将要进行什么运算

③取操作数

如果是ALU运算类指令

则根据IR中指令指定的寄存器编号选择两个或一个寄存器中的内容作为操作数，送到ALU的输入端

④指令执行

若是ALU运算类指令，则ALU在控制信号的控制下，进行相应的运算，并生成运算结果和相应的标志信息

若是取数/存数指令，则主存在读/写控制信号的控制下，对地址线指定的存储单元进行读/写

⑤回写结果

将ALU运算的结果写入指定编号的寄存器或指定的主存单元中



通常在主存取指令时
自动修改PC值为PC+“1”
以实现自动执行程序中的指令

指令的执行由时钟信号定时
一条指令的执行可能需要
一个或多个时钟周期





4. 计算机系统性能评价

4.1 计算机性能指标

吞吐量 指系统在单位时间内处理请求的数量

响应时间 指从用户发出请求到系统返回所需结果的总等待时间
包括**CPU时间**和其他时间

永久微信1931637888

指CPU用于程序执行的时间

CPU时间 包括**用户CPU时间**（真正用于运行用户程序代码的时间）
和**系统CPU时间**（为了执行用户程序而需要CPU运行
操作系统程序的时间）

其他时间 指等待I/O操作完成的时间或CPU用于执行其他用户程序的时间





4. 计算机系统性能评价

4.1 计算机性能指标

CPU时钟周期 机器内部主时钟脉冲的宽度
是CPU工作的最小时间单位

后浪关注公众号「陆大」
永久微信1951657888

时钟周期以相邻状态单元间组合逻辑电路的最大延迟时间为基准确定
在流水线中以指令流水线的每个流水段的最大延迟时间为准

CPU时钟频率（主频） 机器内部主时钟的频率
即CPU时钟周期的倒数

主频表示每秒包含的时钟周期数





4. 计算机系统性能评价

4.1 计算机性能指标

CPI(Cycles Per Instruction) 执行一条指令所需的时钟周期数

后续关注公众号「研大」

对于一台机器/程序来说，其CPI是该程序/机器指令集中所有指令执行所需的平均时钟周期数，称为平均CPI

IPS(Instructions Per Second) 每秒执行多少条指令

$$\text{IPS} = \frac{\text{主频}}{\text{平均CPI}}$$





4. 计算机系统性能评价

4.1 计算机性能指标

CPU执行时间 指运行一个程序所花费的时间

后续关注公众号[研大]
永久微信1951657888

$$\text{CPU执行时间} = \frac{\text{CPU时钟周期数}}{\text{主频}}$$

$$\text{CPU执行时间} = \frac{\text{指令条数} \times \text{CPI}}{\text{主频}}$$





4. 计算机系统性能评价

4.1 计算机性能指标

MIPS (Million Instructions Per Second) 每秒执行多少百万条指令

FLOPS (Floating-point Operations Per Second) 每秒执行的浮点运算次数

永久微信1951657888

MFLOPS (Million FLOPS) 百万 (10^6) 次浮点运算/秒

GFLOPS (Giga FLOPS) 十亿 (10^9) 次浮点运算/秒

TFLOPS (Tera FLOPS) 万亿 (10^{12}) 次浮点运算/秒

PFLOPS (Peta FLOPS) 千万亿 (10^{15}) 次浮点运算/秒

EFLOPS (Exa FLOPS) 百亿亿 (10^{18}) 次浮点运算/秒

ZFLOPS (Zetta FLOPS) 十万亿亿 (10^{21}) 次浮点运算/秒





4. 计算机系统性能评价

4.1 计算机性能指标

MIPS (Million Instructions Per Second) 每秒执行多少百万条指令

FLOPS (Floating-point Operations Per Second) 每秒执行的浮点运算次数

MFLOPS (Million FLOPS) 百万(10^6)次浮点运算/秒

GFLOPS (Giga FLOPS) 十亿(10^9)次浮点运算/秒

TFLOPS (Tera FLOPS) 万亿(10^{12})次浮点运算/秒

PFLOPS (Peta FLOPS) 千万亿(10^{15})次浮点运算/秒

EFLOPS (Exa FLOPS) 百亿亿(10^{18})次浮点运算/秒

ZFLOPS (Zetta FLOPS) 十万亿亿(10^{21})次浮点运算/秒

口诀：美(M)国(G)突(T)破(P)二(E)战(Z)





4. 计算机系统性能评价

4.2 基准程序



- 基准程序是专门用来进行性能评价的一组程序，能够很好地反映机器在运行实际负载时的性能
- 可以通过在不同机器上运行相同的基准程序来比较在不同机器上的运行时间，从而评测其性能
- 基准程序最好是用户经常使用的一些实际程序，或是某个应用领域的一些典型的简单程序
- 对于不同的应用场合，应该选择不同的基准程序





真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 假设某机器M的时钟频率为2GHz, 用户程序P在M上的指令条数为 1×10^9 , 其CPI为1.5, 则P在M上的执行时间是多少? 若在机器M上从程序P开始启动到执行结束所需的时间是2s, 则程序P占用的用户CPU时间的百分比是多少?

微信公众号: 计算机考研
公众号: 1901087000





真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 若机器M的主频为1.5GHz, 在M上执行程序P的指令条数为 5×10^5 , P的平均CPI为1.2, 则P在M上的指令执行速度和用户CPU时间分别为

A. 0.8GIPS, 0.4ms

扫码关注公众号
永久微信1981487888

B. 0.8GIPS, 0.4us

C. 1.25GIPS, 0.4ms

D. 1.25GIPS, 0.4us





祝你今天特别特别开心！

